BUILDCONNECT

A Hyper-Local Home Services Marketplace

23CS54C – MODERN WEB TECHNOLOGIES

Submitted by

[YOUR NAME] [REG NO]

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

OCTOBER - 2026

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "BuildConnect", is the bonafide work of [YOUR NAME] ([REG NO]) who carried out the project work under my supervision.

_____

Course Instructor / Guide

Submitted to the **23CS54C – MODERN WEB TECHNOLOGIES** Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

_____ _____

Internal Examiner **Co Examiner**

TABLE OF CONTENT

CH NO	TITLE	PAGE NO
1	INTRODUCTION	1
2	OBJECTIVES	2
3	DESCRIPTION	3
4	CLASS DIAGRAM	8
5	TABLE STRUCTURES	10
6	ER DIAGRAM	14
7	TECH STACK	16
8	MODULES	19
9	CONCLUSION	54

CHAPTER 1
INTRODUCTION

Every household at some point needs a plumber, an electrician, a painter or a carpenter. Today these jobs are arranged by word of mouth and phone calls: customers have no way to compare tradespeople, no idea what a fair price looks like and no record of the work once it is done. Skilled professionals, on the other hand, depend on personal references to find new work, quote prices verbally and often struggle to track pending jobs, payments and material costs. Informal bookings lead to disputes over price, quality and payment, and both sides lose time, money and trust.

BuildConnect is a full-stack, hyper-local marketplace platform designed to bridge this gap between customers who need home services and verified providers (skilled tradespeople and service businesses) across Tamil Nadu. It serves three types of users – Customer, Provider and Admin – each with a dedicated dashboard and workflow.

The platform follows a booking lifecycle: a customer discovers services by category, city and price, picks a date and time slot, adds an address on an interactive map and places a booking; the provider accepts or rejects the booking, sends an estimate and adds material costs as the work progresses; the customer pays securely through Razorpay and downloads a PDF bill; finally the completed work is rated by the customer. Every transition raises a real-time notification in the app and, subject to the user's communication preferences, an e-mail, SMS or WhatsApp message that is recorded in a delivery log.

The application is organized into three functional domains: Customer features such as service, provider and package discovery with filters, service details with reviews and portfolio, slot-based booking, Razorpay checkout, downloadable invoices, live chat and ratings; Provider features such as profile onboarding with admin approval, service, package and portfolio management, a booking board with estimates and material tracking, bills and analytics; and Admin features such as dashboard metrics, provider approval, content moderation and user management. A set of core engines – booking lifecycle, payments, invoices, real-time messaging, multi-channel notifications, ratings and maps – supports these domains.

The system is built with React 18, Vite and Tailwind CSS on the frontend and Node.js, Express and Socket.IO on the backend, with MongoDB and the Mongoose ODM as the database, and Razorpay, Google OAuth, Nodemailer, Twilio and Leaflet / OpenStreetMap as integrations. The interface ships in English, Hindi and Tamil with light and dark themes and covers city options across all 38 Tamil Nadu districts. This report documents the design, development and implementation of BuildConnect, providing a complete technical overview of its architecture, features and functionality.

CHAPTER 2

OBJECTIVES

- **To Simplify Hiring:** Allow customers to find skilled plumbers, electricians, painters and carpenters, compare prices and book a date in a single flow.
- **To Provide Transparent Pricing:** Publish fixed prices, hourly rates and package quotes up front, and extend them with customer-approved estimates and material costs.
- **To Guarantee Secure Payments:** Capture payments through Razorpay with server-side signature verification and issue a PDF bill for every completed booking.
- **To Deliver a Predictable Booking Lifecycle:** Move each booking through PENDING → ACCEPTED → ESTIMATE_PROVIDED → IN_PROGRESS → COMPLETED with explicit reject and cancel states.
- **To Deliver Real-Time Experience:** Use Socket.IO to broadcast chat messages, booking updates and in-app notifications to both parties instantly.
- **To Verify Service Providers:** Allow administrators to approve every provider account before the provider's listings become bookable.
- **To Enforce Authentication and Role-Based Access:** Secure the three roles with bcrypt, JWT, Helmet and rate limiting, and offer Google OAuth sign-in as a password-free option.
- **To Keep Every Party Informed:** Send lifecycle messages over e-mail, SMS and WhatsApp according to each user's communication preferences, with a delivery log of outcomes.
- **To Build Trust Through Proof of Work:** Showcase provider portfolios, verified reviews and recalculated ratings on services, packages and provider profiles.
- **To Enable Location-Based Service:** Use an interactive Leaflet map to pick booking addresses accurately and to view routes to the service location.
- **To Measure Platform Activity:** Report users, providers, services, bookings by status and reviews to the admin, and booking history to providers through dashboards.
- **To Improve Site Productivity:** Track appointments with time slots, booked-slot availability, estimates and material line items so nothing is forgotten on site.
- **To Make the Platform Inclusive:** Provide an English, Hindi and Tamil interface with light and dark themes that work on both phone and laptop screens.

CHAPTER 3
DESCRIPTION

BuildConnect is a full-stack web application that connects customers, service providers and administrators. The system consists of four major functional areas:

1. Customer (Homeowner / Booking User)

Customers sign in, discover services and providers by category and city, check ratings, prices and portfolios, pick a date and time slot, place a booking with a map-pinned address, pay through Razorpay, follow the booking live, chat with the provider, download the PDF bill and rate the work after completion.

Feature	**Implementation & Path**	**Description**
Authentication & Profile	App.jsx (LoginPage, RegisterPage), authController.js	Sign up / login with JWT, Google OAuth sign-in, profile and password management.
Service Discovery	App.jsx (ServicesPage), serviceController.js	Search and filter services by category, city, price band and availability; sorted by bookings and rating.
Providers & Packages	App.jsx (ProvidersPage, PackagesPage), packageController.js	Browse approved providers, fixed-price packages with duration, experience and ratings.
Details & Reviews	App.jsx (ServiceDetailsPage, ProviderProfilePage), reviewController.j	Full details with pricing, portfolio images and reviews for a service, provider or package.

Feature	Implementation & Path	Description
	s	
Slot-Based Booking	App.jsx (BookingPage), bookingController.js	Date, time slot, live booked-slot check, address with map picker, requirements or custom quote.
Payment & Bill	paymentController.js, invoiceService.js	Razorpay checkout with signature verification and a downloadable INV-XXXXXXXX PDF bill.
Live Chat	messageController.js, Socket.IO	Real-time per-booking chat room with offline notification fallback.
Saved Items & Alerts	App.jsx (SavedPage, NotificationBell), favorites.js	Favourites list plus an in-app notification bell with unread count.
Ratings & Feedback	reviewController.js	Post-completion 1–5 rating for the provider, service or package with instant average recalculation.

2. Provider (Skilled Professional / Service Business)

Providers register with their trade and store details, wait for administrative approval, publish services and packages with prices, showcase past work in a portfolio, and run accepted bookings through estimates, material entries and status changes.

Feature	Implementation & Path	Description
Provider Onboarding	App.jsx (ProviderOnboardingTour),	Sign-up with trade, store and address details; admin approval gate before listings go live.

Feature	Implementation & Path	Description
	userController.js	
Service Catalog	App.jsx (ProviderDashboard), serviceController.js	Create, edit or delete services with category, price, hourly rate, city, experience and photos.
Service Packages	packageController.js	Fixed-price packages with duration, description, availability and booking counts.
Portfolio Showcase	portfolioController.js	Upload past work with photos and optionally tag the customer and booking it belongs to.
Booking Board	App.jsx (ProviderDashboard), bookingController.js	Accept or reject requests, move bookings through the status flow and cancel when required.
Estimates & Materials	bookingController.js (provideEstimate, addMaterials)	Send an estimate, then add material / expense line items that extend the bill total.
Bills & Payments	invoiceService.js, paymentController.js	Preview and download the PDF bill; track the PAID status of each booking.
Chat & Notifications	Socket.IO, notificationController.js	Answer customers in real time and receive booking and message alerts.

3. Admin (Platform Overseer)

The administrator monitors platform activity, approves provider accounts, moderates content and manages users.

Feature	Implementation & Path	Description
Admin Dashboard	App.jsx (AdminDashboard), adminRoutes.js	Live counts of users, providers, customers, services, bookings by status and reviews.
Provider Approval Workflow	adminRoutes.js (approve-provider)	Review and approve provider accounts before their services become bookable.
User & Content Management	adminRoutes.js	List and delete users, services and packages; remove reviews that violate policy.
Bookings Oversight	adminRoutes.js (bookings)	All bookings across the platform with customer, provider and service details populated.

4. Core System Engines & Intelligence

- **Booking Lifecycle Engine:** A seven-state status flow (PENDING, ACCEPTED, ESTIMATE_PROVIDED, IN_PROGRESS, COMPLETED, REJECTED, CANCELLED) with timestamps for acceptance, start, completion and cancellation.
- **Razorpay Payment Engine:** Server-created orders priced in paise, HMAC-SHA256 signature verification before a payment is marked PAID, and refund-capable payment records.
- **PDF Invoice Engine:** pdfkit bills numbered INV-XXXXXXXX whose grand total equals the listed price plus all material / expense entries, including the amount in words.
- **Real-Time Engine:** Socket.IO rooms – one per booking for chat and one per user for notifications – so messages and alerts reach the right screens instantly.

- **Multi-Channel Notification Engine:** Lifecycle messages delivered over e-mail (Nodemailer), SMS and WhatsApp (Twilio, with a Meta option) with a per-event delivery log of SENT, FAILED or SKIPPED outcomes.
- **Review & Rating Engine:** Rating averages recomputed on the provider and the service after every review or deletion.
- **Media Engine:** Multer handles profile, service and portfolio uploads served from the /uploads static route.
- **Maps, Theme & Language:** Leaflet location picker and route viewer, light / dark / system theme, and a searchable language switch covering English, Hindi and Tamil.

Technology and Deployment. The frontend uses React 18 with Vite and Tailwind CSS, the backend uses Node.js, Express and Socket.IO, and MongoDB with the Mongoose ODM stores the data. Security is provided by bcrypt, JWT, Helmet, express-rate-limit and Zod validation, while Razorpay handles payments and Nodemailer and Twilio handle transactional communication. The modular architecture supports maintainability, scalability and future enhancements.

CHAPTER 4
CLASS DIAGRAM

[Insert image here]

Fig 4.1: Class diagram for BuildConnect

The BuildConnect Class Diagram represents the overall structure and functionality of the BuildConnect application by showing its major classes, their attributes, methods and relationships. The system is divided into six main modules: User & Authentication, Catalog (Service, Package & Portfolio), Booking & Estimate, Payment & Invoice, Message & Notification, and Admin & Review. The User and Authentication module manages registration, login, Google sign-in, roles, profile data, city and trade information, availability, approval flags and communication preferences. The Catalog module manages service listings with category, price, hourly rate and experience, fixed-price packages and portfolio works with tagged customers and bookings. The Booking and Estimate module handles customer bookings, time slots, addresses, requirements, custom quotes, estimates,

material entries and the full status flow. The Payment and Invoice module covers Razorpay orders, signature verification, payment status and PDF bill generation with sequential invoice numbers. The Message and Notification module provides per-booking chat, in-app notifications, read states and e-mail, SMS and WhatsApp delivery logs. The Admin and Review module allows administrators to approve providers, manage users and content, while customers publish reviews that recalculate provider and service ratings. The diagram also uses enumerations such as RoleType, BookingStatus, PaymentStatus, ServiceCategory, NotificationChannel, DeliveryStatus and ThemeMode to maintain consistent values throughout the application. Overall, the class diagram shows how customers, providers, services, packages, bookings, payments, messages, notifications, reviews and administrators interact to provide a complete hyper-local home services marketplace.

The BuildConnect Class Diagram represents the static structure of the BuildConnect application. It shows the major classes, their attributes, methods, relationships and enumerations used to implement the system.

The system is divided into six major modules:

1. User & Authentication Module
2. Service, Package & Portfolio Catalog Module
3. Booking & Estimate Module
4. Payment & Invoice Module
5. Message & Notification Module
6. Admin & Review Module

The diagram also contains enumerations that define fixed values for user roles, booking status, payment status, service categories, notification channels, delivery outcomes and theme modes.

CHAPTER 5
TABLE STRUCTURES

The backend stores its data in MongoDB collections defined by Mongoose schemas. Each schema below describes the key fields, types and references (ObjectId links between collections) of the nine collections of the application. Embedded documents – the material / expense entries and the message delivery log of a booking – live inside their parent booking record and are therefore listed with it.

bookings

[Insert collection / schema screenshot]

messages

[Insert collection / schema screenshot]

notifications

[Insert collection / schema screenshot]

packages

[Insert collection / schema screenshot]

payments

[Insert collection / schema screenshot]

portfolioworks

[Insert collection / schema screenshot]

reviews

[Insert collection / schema screenshot]

services

[Insert collection / schema screenshot]

users

[Insert collection / schema screenshot]

CHAPTER 6
ER DIAGRAM

The BuildConnect ER Diagram represents the complete database architecture of the single MongoDB database used by the application. The users collection holds customers, providers and administrators, distinguished by the role field, together with approval, rating, address and communication-preference data. Services, packages and portfolio works reference their provider; bookings reference the customer, provider, service and package; payments reference the booking, customer and provider; reviews reference the customer, provider, service, package and booking; messages reference the booking with sender and receiver users; and notifications reference their recipient. ObjectId references and schema validations maintain data integrity across these relationships, while embedded arrays keep a booking's material entries and communication history transactionally together with it. The database supports the complete service lifecycle, from provider approval and service publication to booking, estimate, payment, invoicing, review and administrative monitoring. Overall, the ER diagram provides a structured, secure and scalable data design for the BuildConnect application.

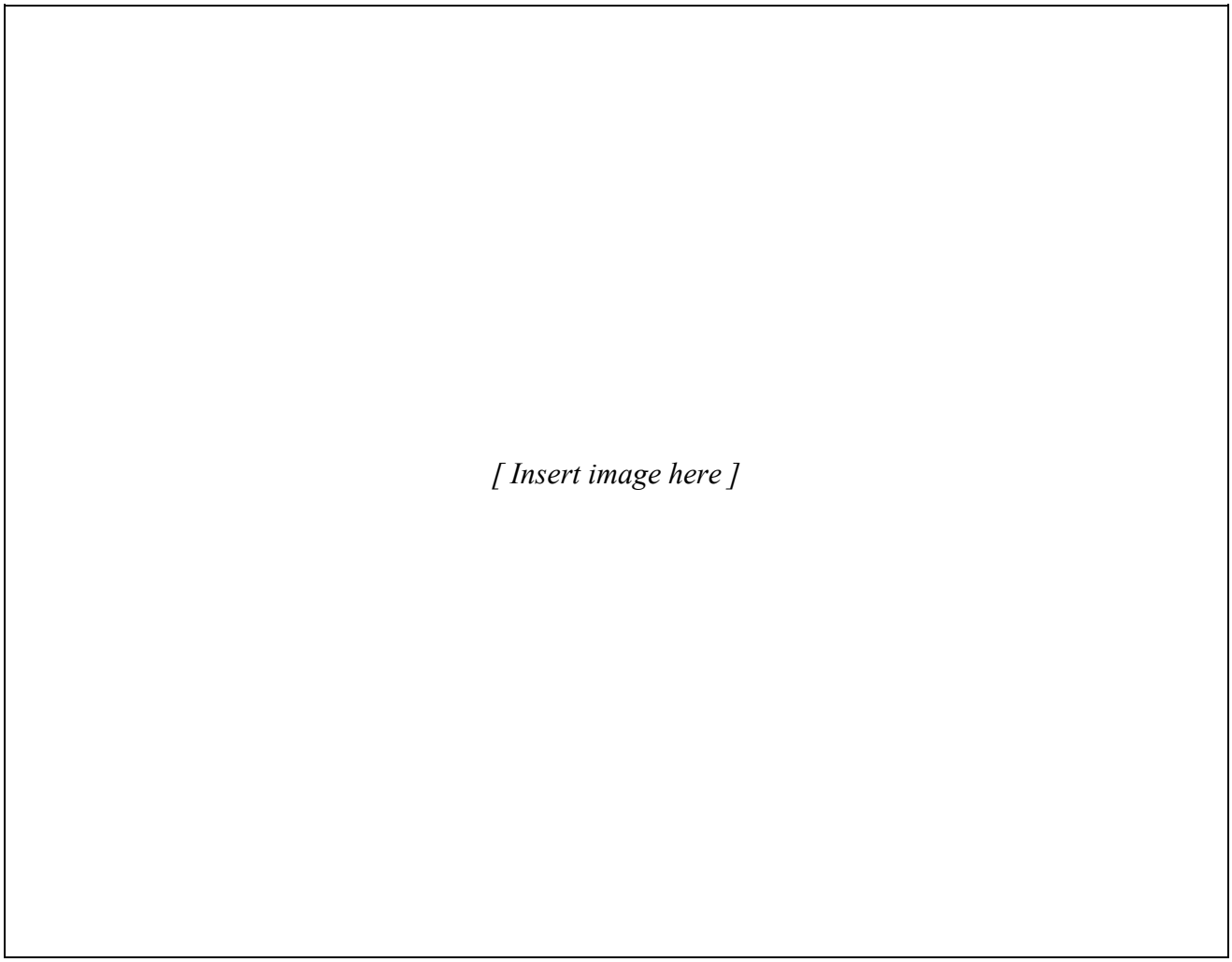


Fig 6.1: ER diagram for BuildConnect

CHAPTER 7
TECH STACK

7.1 TECH STACK

Layer	Technology	Purpose
Frontend	React 18 + Vite	Single-page application with fast HMR and production builds
Styling	Tailwind CSS 4	Utility-first responsive UI in light and dark themes
UI Components	Radix UI (shadcn-style, 50+ components)	Accessible dialogs, dropdowns, forms, tables and toasts
Data Client	Axios + TanStack React Query	REST calls, caching and background refetching
State	React Context (App context)	Session, navigation, toasts and route data across screens
Charts	Recharts	Admin and provider dashboard visuals
Icons	Lucide React	Consistent icon set
Maps	Leaflet + OpenStreetMap	Location picker, address search and route viewer
Internationalisation	Google Translate engine + LanguageSelector	English, Hindi and Tamil interface
Backend	Node.js + Express 5	RESTful API server (11 route files)
Real-time	Socket.IO	Per-booking chat rooms and per-user notification rooms
Database	MongoDB + Mongoose	Document storage with references,

Layer	Technology	Purpose
	ODM	validations and timestamps
Authentication	bcryptjs + JSON Web Tokens + Google OAuth	Password hashing, stateless sessions and Google sign-in
Security	Helmet, express-rate-limit, Zod	HTTP headers, abuse control and request validation
Payments	Razorpay	Online payments with order creation and signature verification
Invoices	pdfkit	Server-generated PDF bills with sequential invoice numbers
E-mail	Nodemailer	Transactional e-mail for booking lifecycle events
SMS / WhatsApp	Twilio (Meta option)	SMS and WhatsApp lifecycle messages
File Uploads	Multer	Profile, service and portfolio image uploads
Testing	Jest + Supertest / Vitest + Testing Library	Backend API tests and frontend component tests

7.2 ARCHITECTURE OVERVIEW

Layer	Technology	Purpose
Model	Mongoose schemas + MongoDB (config/db.js, models/)	Defines the data structure, handles queries, references and validations.
View	React 18 (App.jsx screens, 50+ UI components)	Renders the customer, provider and admin dashboards and communicates

Layer	Technology	Purpose
		with the server through the API client and Socket client.
Controller	Node.js + Express controllers and routes	Receives requests, applies business logic (Auth, Bookings, Payments, Messages, Admin) and coordinates between Model and View.

CHAPTER 8
MODULES

Shared baseline for all modules

Non-functional baseline. Every request travels through Helmet (security headers), the rate limiter (100 requests per IP per 15 minutes) and the JWT authentication middleware before it reaches a controller. All data lives in one MongoDB database, so the system chooses consistency over availability in CAP terms (CP): a customer may briefly be unable to book if the database is unreachable, but two customers can never hold the same slot unawares, a payment is marked PAID only after a verified signature and a booking status only moves forward through the defined sequence. Live screens are kept current by Socket.IO.

Sizing baseline

All figures below are assumptions for a growing district-level deployment; the others follow from them.

Quantity	**Estimate**	**Basis**
Accounts	≈ 500	Assumption: customers, providers and admins
Providers / services / cities	100 / 300 / 40	Assumption: catalogue size after one year
Daily active users	≈ 50	Assumption: about 10% of accounts
Requests per active user per day	≈ 8	Assumption: feeds, details, booking status, chat, notifications
Total requests per day	50 × 8 = 400	Derived
Average load	400 ÷ 57,600 s ≈ 0.01 req/s	Spread over a 16-hour usage day
Peak load	≈ 1–3 req/s	Evening booking rush, roughly 100× the average
Real-time connections	≈ 50–150	One Socket.IO connection per open

Quantity	Estimate	Basis
		app

A single Node.js process and one MongoDB instance handle this load comfortably. The API is stateless (JWT instead of server sessions), so extra instances can be added behind a load balancer; running several Socket.IO instances would additionally require a shared adapter such as the MongoDB or Redis connector.

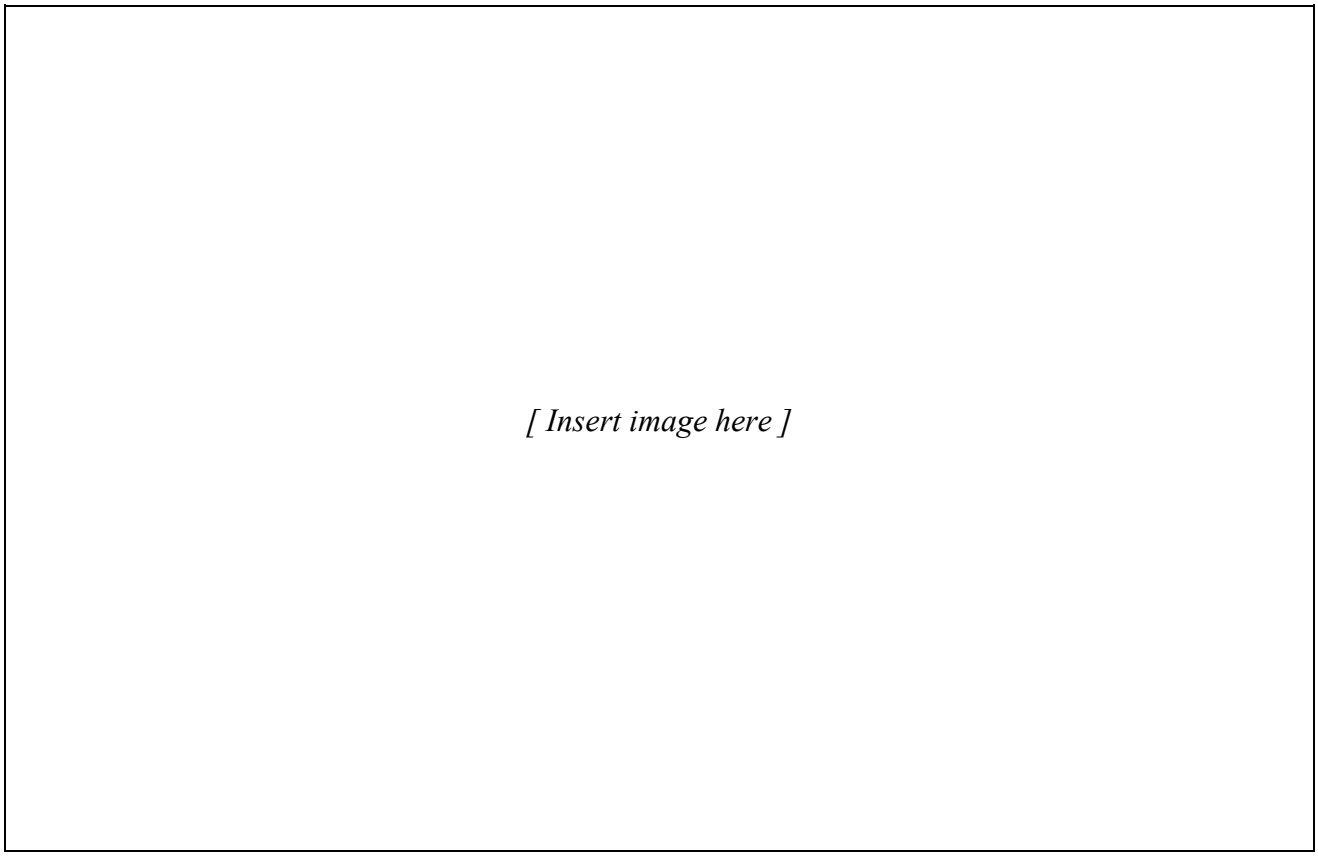


Fig 8.1: Overall request flow of BuildConnect

8.1 FOLDER STRUCTURE

The project is split into a backend API server and a frontend single-page application.

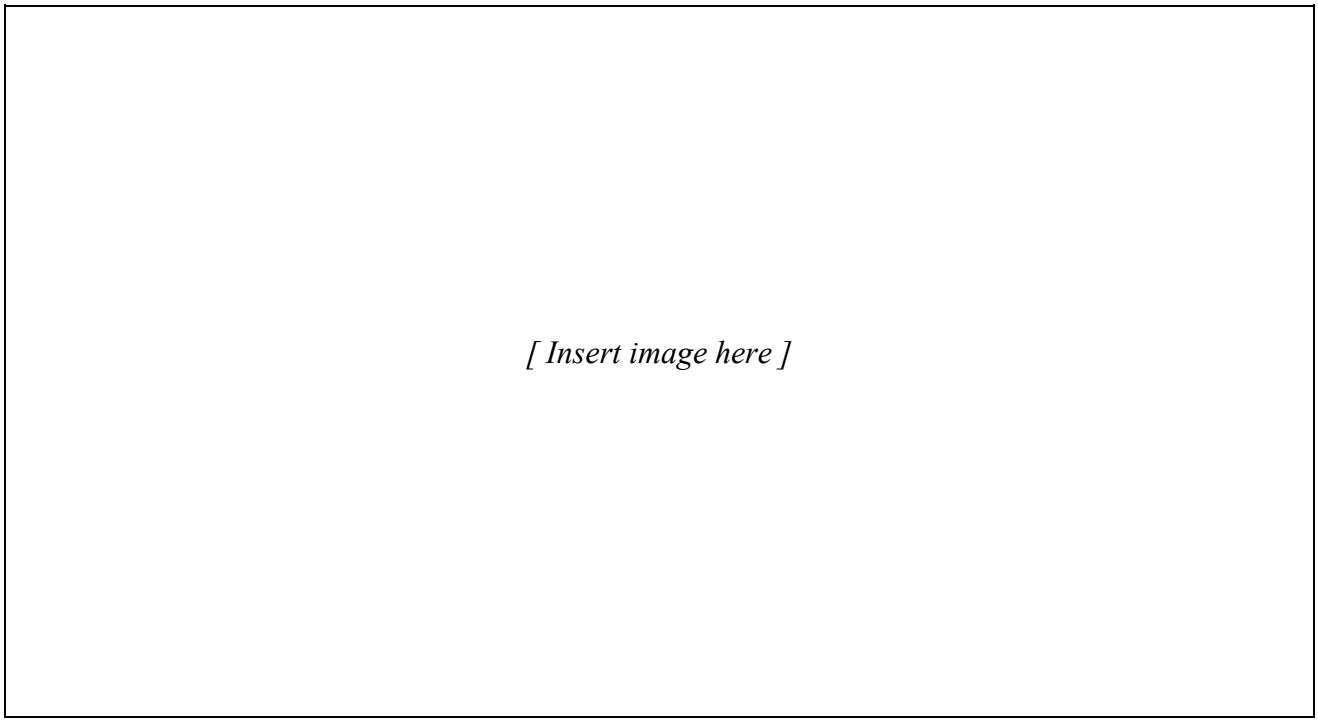


Fig 8.2: Project folder structure

BACKEND FOLDER

- config: database connection (db.js) and environment settings, so credentials stay out of the code.
- controllers: the business logic handlers – Auth, Bookings, Services, Packages, Portfolio, Payments, Messages, Notifications, Reviews, Users and Admin.
- models: nine Mongoose schemas – User, Service, Package, PortfolioWork, Booking, Payment, Review, Message and Notification.
- middleware: JWT authentication (protect / authorize) and Multer upload handling.
- routes: REST endpoint definitions in 11 route files mounted under /api.
- services: pdfkit invoice generation and the multi-channel notification service (e-mail, SMS, WhatsApp).
- scripts, seedAdmin.js, tests: sample-bill generator, admin seeding and Jest + Supertest API tests.

FRONTEND FOLDER

- app/App.jsx: all customer, provider and admin screens with a context-based app shell.

- app/components/ui: 50+ accessible Radix-style UI components (dialog, dropdown, table, toast …).
- components: LanguageSelector, MapLocationPicker, MapRouteViewer, ThemeProvider and ThemeToggle.
- services: Axios API client, favourites helper; styles: Tailwind theme and fonts; test: Vitest setup.

8.2 AUTHENTICATION, GOOGLE OAUTH & ROLE-BASED ACCESS

Purpose & Scope

Provides sign-up, login, Google sign-in, password management and profile management for the three user roles of the platform – Customer, Provider and Admin – using JSON Web Tokens (JWT), so that every dashboard and API route is reachable only by the role that is allowed to use it.

Functional Requirements

What the module must do:

- A user registers or logs in (RegisterPage, LoginPage) and receives a signed JWT; the client sends it as a Bearer token on every later request.
- Google OAuth sign-in verifies the ID token server-side (google-auth-library) and creates or reuses the account (authController.googleLogin).
- A signed-in user can manage the profile, avatar, city and address details and change the password (GeneralSettingsPanel, userController).
- Three roles – customer, provider, admin – decide which dashboard is shown and which API routes may be called (middleware authorize).
- Provider accounts carry an isApproved flag, so an unapproved provider cannot be booked until an administrator approves them.
- Protected routes redirect unauthenticated users to the login screen; an expired or invalid token is handled centrally by the Axios client.

Out of scope: multi-factor authentication and role hierarchies beyond the three roles.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Security:** passwords are stored only as bcryptjs hashes with select: false, so the hash never leaves the database layer.
- **Brute-force defence:** express-rate-limit throttles the API per client address; Helmet adds secure HTTP headers.
- **Statelessness:** JWT lets any server instance validate a request without a shared session store.
- **Latency target:** login completes in well under one second; token verification adds only microseconds per request.
- **CAP:** the users collection is a single MongoDB source of truth – consistency is preferred, so a disabled or unapproved account is honoured on the very next request.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
POST	/api/auth/register (public)	Creates an account with a hashed password and a role (customer by default; providers then wait for approval).
POST	/api/auth/login (public)	Body: e-mail, password. Returns the JWT and the user profile. Rate limited.
POST	/api/auth/google (public)	Body: Google ID token. Verifies the token and returns the JWT for the Google account.
GET, PUT	/api/users/profile	Reads and updates name, phone, city, trade, address, theme and communication preferences.
POST	/api/users/change-password	Body: current and new password. Stores a new bcrypt hash.
GET	/api/users/providers (public)	Lists approved providers for discovery screens.

DB Design

Collection	Keys / references	Key fields
users	unique email; sparse unique googleId; role index	name, email, password (select false), googleId, phone, city, trade, role (customer / provider / admin), storeName, address, isApproved, rating, communicationPreferences, theme, timestamps

- The unique index on e-mail makes the login look-up a single index read and prevents duplicate accounts.
- Google accounts are linked through the sparse unique googleId, so password and Google logins can share one profile.

Screenshots

[Insert image here]

Fig 8.3: Login & Sign-up with Google

The login page lets a user sign in with e-mail and password or through Google; successful authentication opens the dashboard that belongs to the user's role.

[Insert image here]

Fig 8.4: Profile & Appearance Settings

8.3 SERVICE, PACKAGE & PORTFOLIO CATALOG

Purpose & Scope

Lets providers publish what they sell – individual services with price and hourly rate, fixed-price packages with duration, and a portfolio of past work – after the administrator has approved the account.

Functional Requirements

What the module must do:

- A provider creates, edits or deletes services (serviceController) with category, name, description, price, hourly rate, city, experience, availability and photos.
- Categories are constrained (Plumbing, Electrical, Painting, Carpentry and custom quotes), so discovery stays consistent.
- Packages (packageController) add fixed-price offers with duration, description, availability, rating and booking counts.
- Portfolio works (portfolioController) showcase past jobs with photos and can tag the customer and booking they came from.
- Uploads are stored through Multer and served from the /uploads static route.

- Listings show rating and bookingsCount so customers can judge popularity at a glance.

Out of scope: multi-branch company accounts and bulk catalogue import.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Data integrity:** only the owning provider (by JWT role and user id) can modify or delete their own services, packages and portfolio items.
- **Usability:** the provider dashboard keeps catalogue management on one screen with instant feedback.
- **Performance:** discovery queries filter on category, city and price and sort by bookingsCount and rating.
- **Media safety:** uploads are restricted by the upload middleware before they reach the catalogue.
- **CAP:** MongoDB is the single source of truth – consistency over availability.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
GET	/api/services (public)	Query: search, category, city, minPrice, maxPrice, availability. Returns populated provider summaries, sorted by bookings and rating.
GET, POST, PUT, DELETE	/api/services[/my|/:id]	Catalogue management for the signed-in provider.
GET, POST, PUT,	/api/packages[/my|/:id]	Fixed-price package management for the signed-in provider.

Method	Endpoint	Purpose / contract
DELETE		
POST, GET, DELETE	/api/portfolio	Create, list by provider (/provider/:providerId) and delete portfolio works.
GET	/api/services/:id (public)	One service with its provider, reviews and packages.

DB Design

Collection	Keys / references	Key fields
services	ref provider → users; index on category / city / price	provider, category, name, description, price, hourlyRate, city, availability, rating, experience, bookingsCount
packages	ref provider → users	provider, title, category, description, price, duration, availability, rating, bookingsCount
portfolioworks	refs provider, taggedCustomer, taggedBooking	title, description, images, tagged customer and booking

- A provider owns many services, packages and portfolio works (1 : N each).
- bookingsCount and rating are maintained on the catalogue so listing screens never aggregate on the fly.

Screenshots

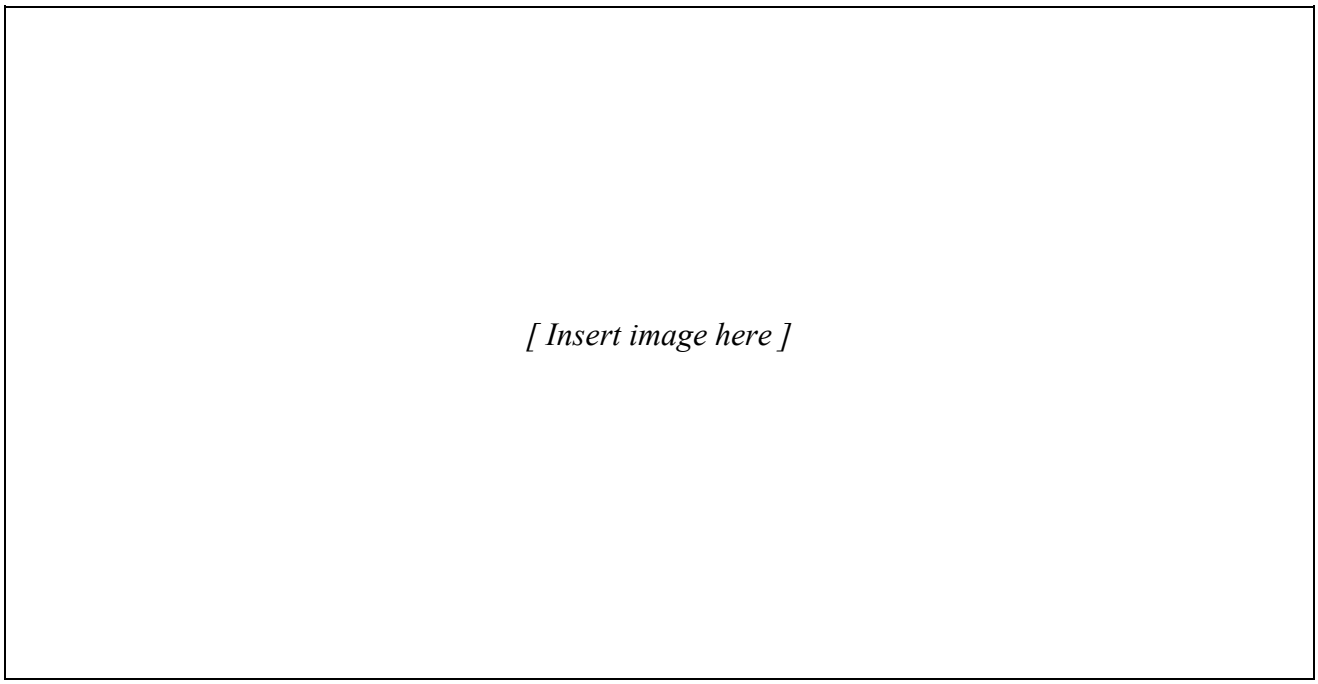


Fig 8.5: Provider Service Management

The provider dashboard lets a provider add, edit or remove services with price, category and photos.

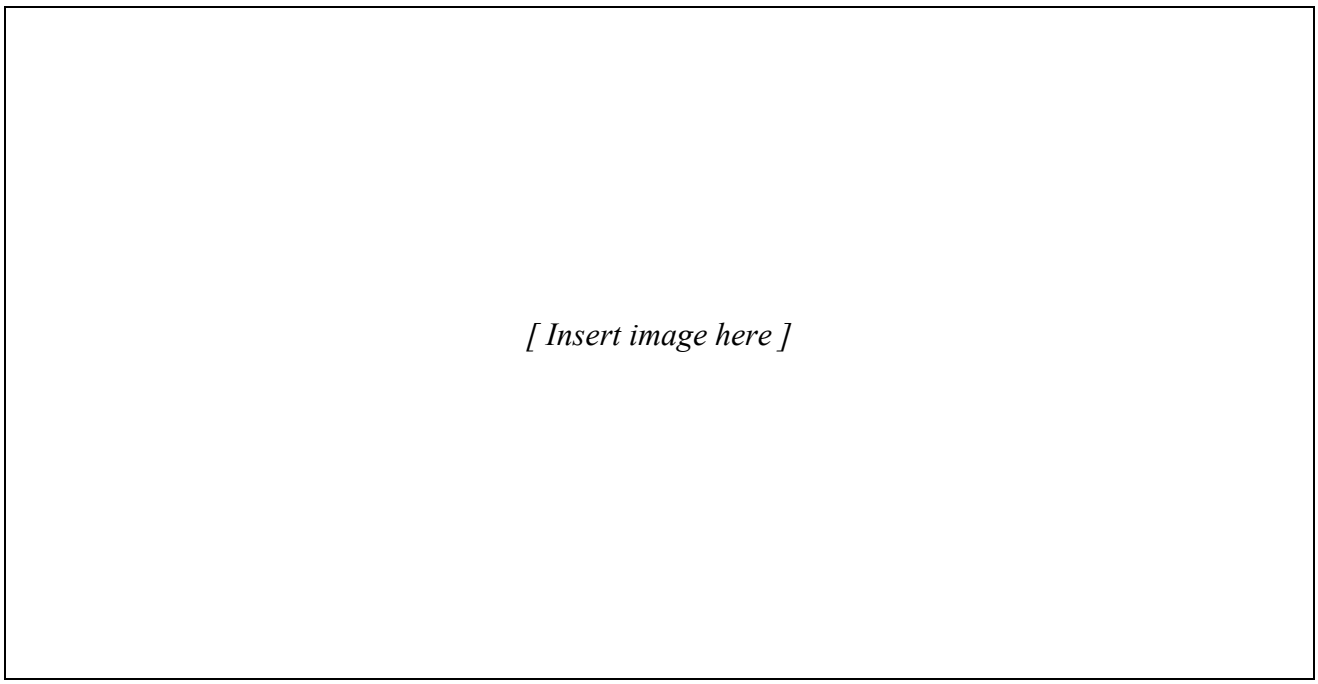


Fig 8.6: Package & Portfolio Manager

8.4 DISCOVERY, DETAILS & MAP INTERACTION

Purpose & Scope

Helps customers find the right professional quickly: text search and filters over services, browsable provider and package listings, rich detail pages with reviews, and interactive maps for pinning and viewing service locations.

Functional Requirements

What the module must do:

- Filter services by search text, category, city, price band and availability in ServicesPage; results are sorted by bookings and rating.
- ProvidersPage and PackagesPage list approved providers and their fixed-price offers with rating, experience and duration.
- Detail pages (ServiceDetailsPage, ProviderProfilePage, PackageDetailsPage) show pricing, portfolio images, availability and reviews for the service, provider or package.
- MapLocationPicker pins the booking address with an address search; MapRouteViewer shows the route to the location.
- SavedPage keeps the customer's favourites for quick return; the AuthGateModal prompts guests to sign in before booking.

Out of scope: delivery-driver style live tracking – providers reach the customer's pinned address.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Responsiveness:** listing pages render the first results quickly on a mobile connection through React Query caching.
- **Accuracy:** booking addresses are pinned as latitude / longitude on the map rather than typed free-form.
- **Read-heavy:** discovery is read-only, so it is safe to cache and cannot corrupt a booking.
- **Compatibility:** the layout works on phone and laptop screens, in light and dark themes.
- **CAP:** reads come from the same MongoDB database; the catalogue changes are visible immediately after a provider edit.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
GET	/api/services (public)	Query: search, category, city, minPrice, maxPrice, availability. Filtered, sorted service list.
GET	/api/users/providers (public)	Approved providers with rating, trade and city.
GET	/api/packages (public)	All packages with provider summaries.
GET	/api/reviews/provider/:providerId	Reviews of one provider for detail pages.
GET	/api/reviews/service/:serviceId	Reviews of one service for detail pages.

DB Design

Collection	Keys / references	Key fields
services	indexes on category, city, price, bookingsCount	Read for filtered listing screens and sorting
users	role + isApproved filter	Read for provider cards, maps and profiles
reviews	refs provider / service / package	Read for detail pages; averages pre-aggregated

- No new collection is created; this module is read-only over the schema.
- Map search and routing are served by OpenStreetMap through the Leaflet components on the client.

Screenshots

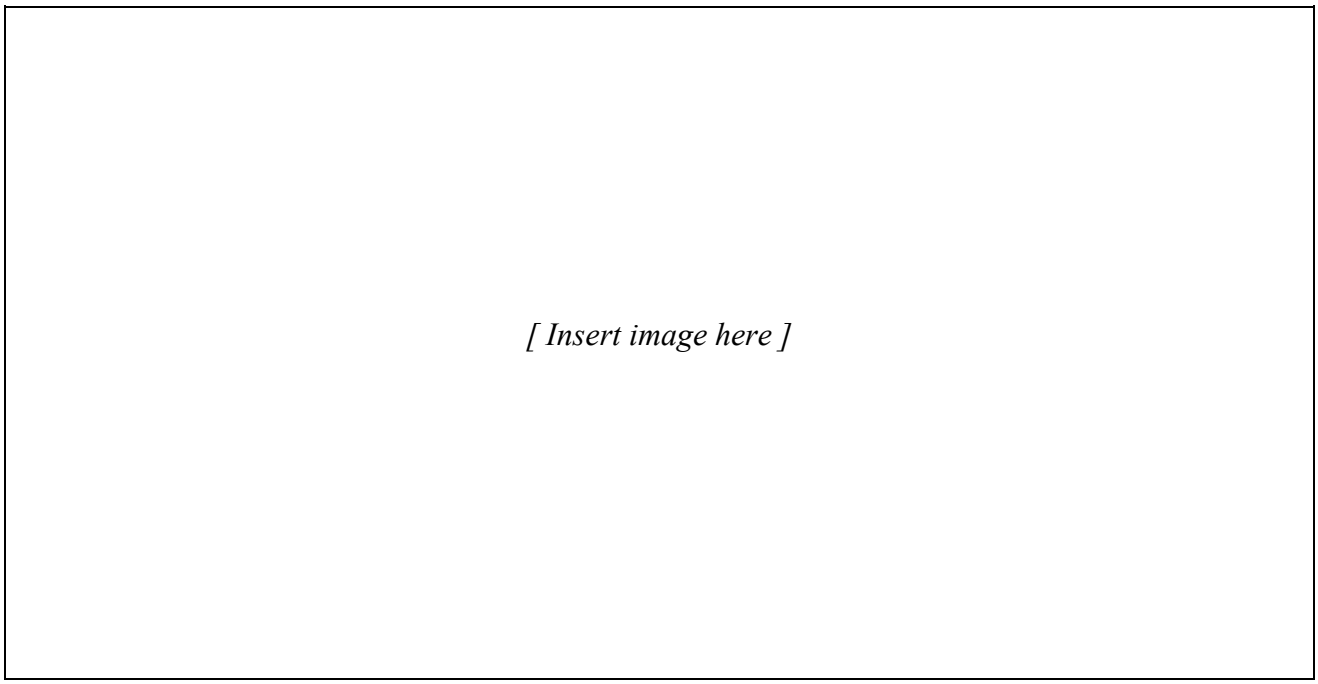


Fig 8.7: Service Discovery with Filters

The customer picks a category and city, sets a price band and compares services sorted by popularity and rating.

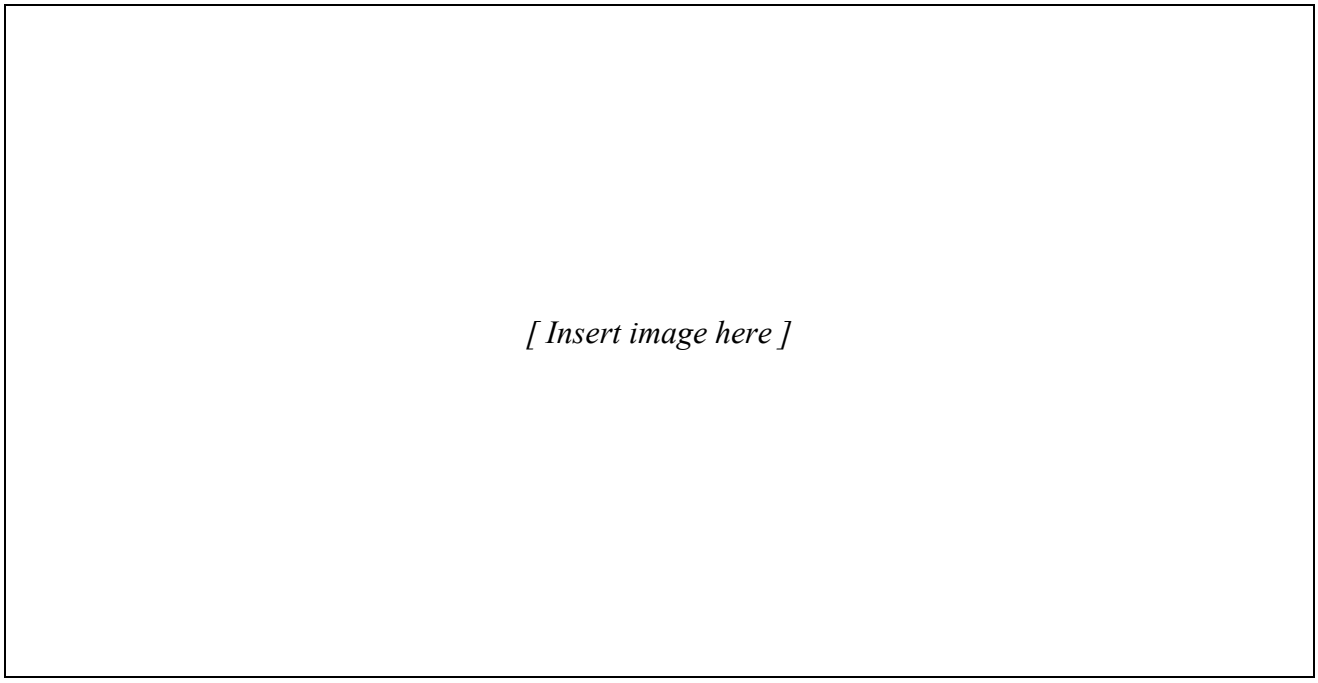


Fig 8.8: Service Details & Reviews

Detail pages show price, availability, portfolio photos and customer reviews before booking.

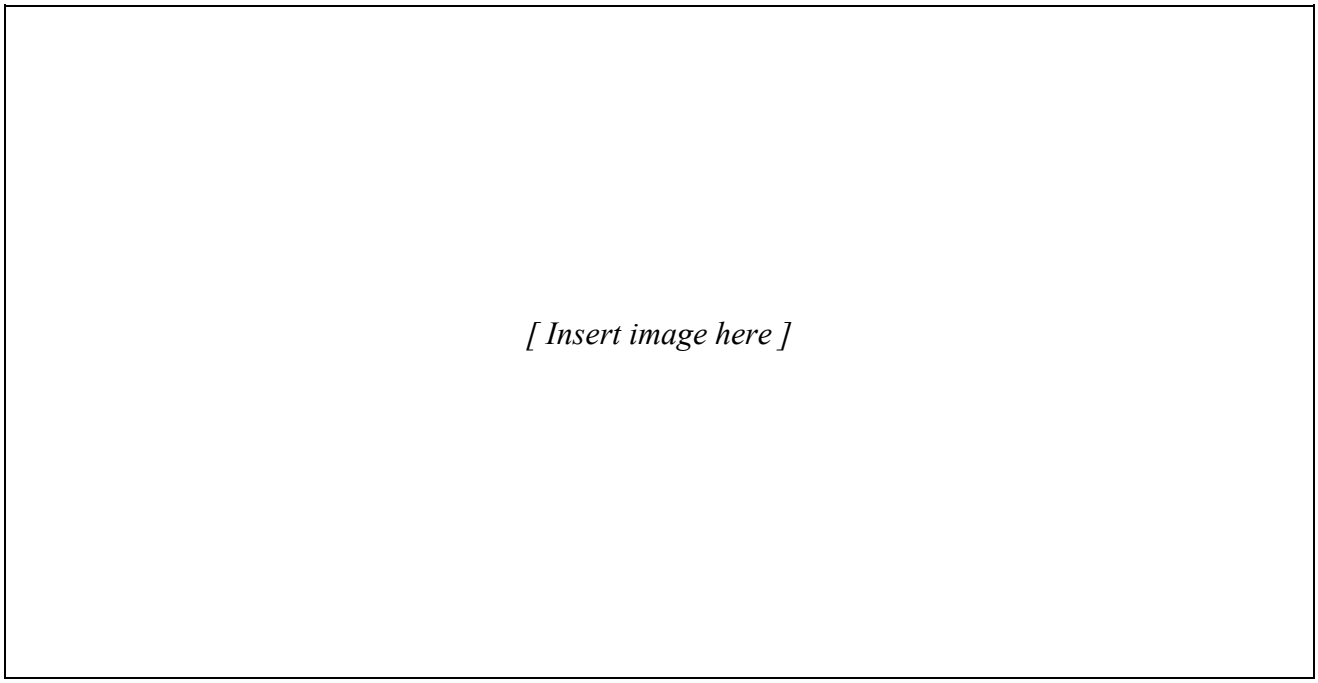


Fig 8.9: Map Location Picker & Route Viewer

8.5 BOOKING LIFECYCLE & ESTIMATES

Purpose & Scope

Turns a customer's selection into a scheduled, trackable job. Bookings carry the date, time slot, map-pinned address and requirements, then move forward through a fixed status flow in which the provider accepts, estimates and completes the work.

Functional Requirements

What the module must do:

- BookingPage creates a booking for a service or package (or a custom-quote request) with date, time slot, address, requirements and price.
- Booked slots are exposed (GET /provider/:providerId/slots) so the calendar never double-books a slot.
- The provider accepts or rejects the booking, provides an estimate (PUT /:id/estimate) and adds material / expense entries (PUT /:id/materials).
- Status moves only forward through PENDING → ACCEPTED → ESTIMATE_PROVIDED → IN_PROGRESS → COMPLETED, with REJECTED and CANCELLED exits; each transition is timestamped.

- Every transition updates a WorkTimeline on both dashboards and triggers the notification engine (Section 8.7).

Out of scope: recurring contracts and subscription scheduling.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Correctness:** slot availability is computed from existing bookings, so the same slot cannot be handed to two customers.
- **Integrity:** status transitions are validated in the controller – a booking cannot skip states or move backwards.
- **Auditability:** acceptedAt, startedAt, completedAt and cancelledAt record when each step happened.
- **Usability:** customers and providers see the same lifecycle message (buildLifecycleMessage) on every change.
- **CAP:** consistency preferred – a booking exists only while MongoDB says so; duplicates are refused.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
POST	/api/bookings	Body: service/package, date, time, address, requirements, price. Creates a PENDING booking.
GET, GET	/api/bookings/my, /api/bookings/provider	Customer and provider views of their bookings, populated for the dashboards.
GET	/api/bookings/provider/:providerId/slots	Booked slots of a provider for a date, used by the slot picker.
PUT	/api/bookings/:id/status	Moves a booking to the next valid status with its timestamp.

Method	Endpoint	Purpose / contract
PUT, PUT	/api/bookings/:id/estimate, /:id/materials	Provider estimate and material / expense line entries.
PUT, DELETE	/api/bookings/:id/cancel, /api/bookings/:id	Cancellation by either party and hard removal of a booking.

DB Design

Collection	Keys / references	Key fields
bookings	refs customer, provider, service, package; index on provider + date	isCustomQuote, customCategory, date, endDate, time, address, requirements, price, status (PENDING … CANCELLED), paymentStatus (UNPAID / PAID), materials[], communicationLogs[], invoiceNumber, lifecycle timestamps

- A booking embeds its materials and delivery log, so an invoice always travels with its data.
- Status and payment status are enums, which keeps invalid states impossible at database level.

Screenshots

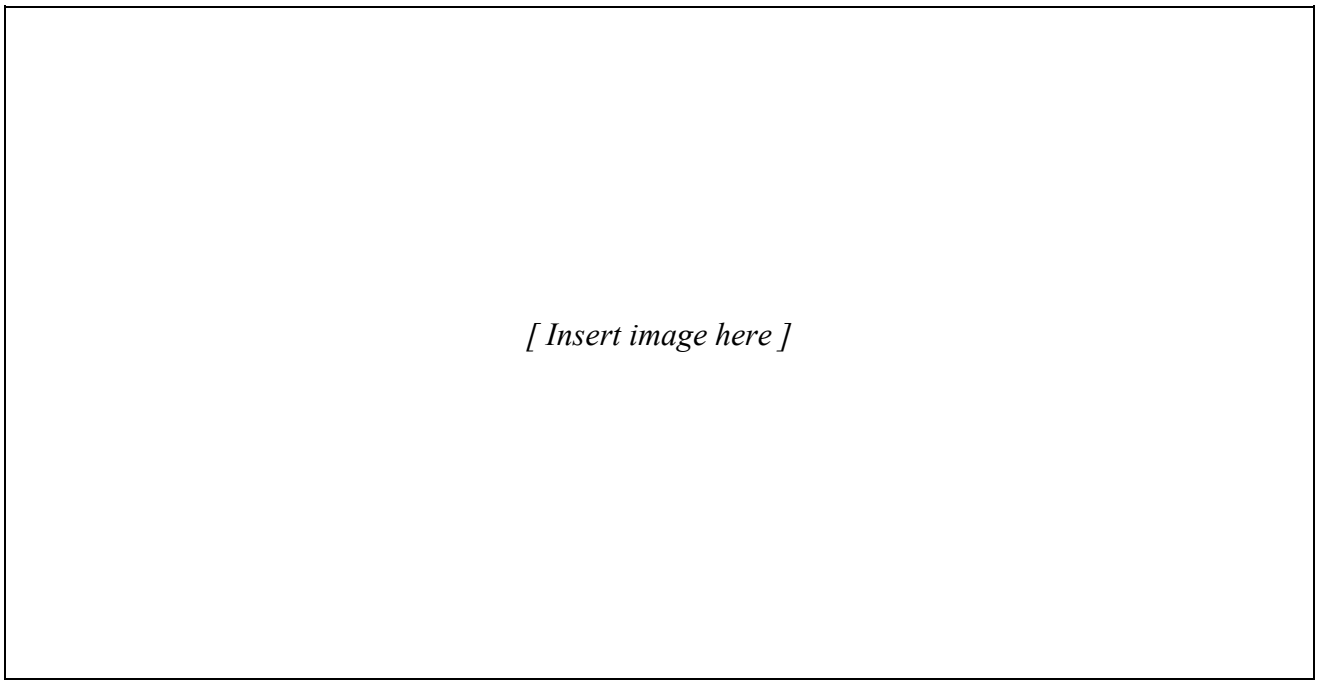


Fig 8.10: Booking Form with Slot Picker

The customer chooses date and time slot, pins the address on the map and submits requirements or a custom quote.

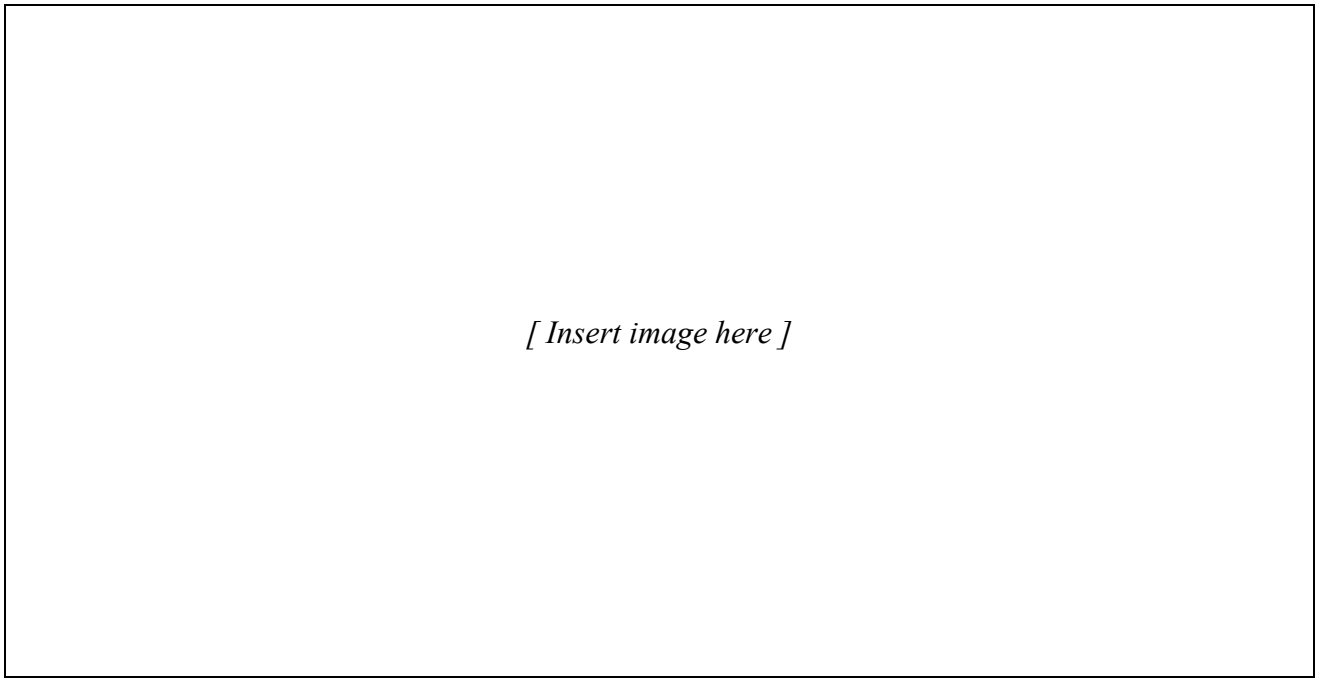


Fig 8.11: Booking Lifecycle Timeline

Both parties follow the same timeline from request to completion, with estimates and material entries inline.

8.6 RAZORPAY PAYMENTS & PDF INVOICES

Purpose & Scope

Handles money and paperwork. The payment engine creates a Razorpay order for the booking amount, verifies the payment signature on the server before marking it PAID, and the invoice engine renders a numbered PDF bill whose total matches what was charged.

Functional Requirements

What the module must do:

- POST /api/payments/create-order creates a Razorpay order with the amount converted to paise and stores a CREATED payment record.
- POST /api/payments/verify recomputes the HMAC-SHA256 signature of order|payment and only on a match marks the payment PAID (booking paymentStatus → PAID).
- The booking sequence assigns invoice numbers (INV-XXXXXXXX) and records invoiceGeneratedAt when a bill is produced.
- invoiceService builds the pdfkit bill: listed price plus every material / expense line equals the grand total, shown with the amount in words and optional GSTIN company details.
- Customers and providers can preview and download the bill (/:id/invoice/preview and /:id/invoice); payment history is available through GET /api/payments/my.

Out of scope: refunds executed from the app UI (records carry a REFUNDED state) and wallet / EMI options.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Payment security:** the signature is verified on the server; card data never touches the platform and the Razorpay key secret stays in environment variables.
- **Atomicity:** the payment record and the booking's paymentStatus change together or not at all.
- **Reconciliation:** razorpayOrderId, razorpayPaymentId and signature are stored on the payment document.
- **Consistency:** bill totals are computed from the same fields the payment charges, so a bill can never disagree with a receipt.

- **CAP:** consistency over availability – under a database fault the payment is refused rather than risking an unrecorded charge.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
POST	/api/payments/create-order	Body: booking. Creates the Razorpay order and a CREATED payment record.
POST	/api/payments/verify	Body: razorpayOrderId, razorpayPaymentId, razorpaySignature. Verifies the signature and marks the payment PAID.
GET	/api/payments/booking/:bookingId	Payment state of one booking.
GET	/api/payments/my	The signed-in user's payment history.
GET	/api/bookings/:id/invoice	Streams the generated PDF bill.
GET	/api/bookings/:id/invoice/preview	Bill data preview before download.

DB Design

Collection	Keys / references	Key fields
payments	refs booking, customer, provider; index on razorpayOrderId	amount, currency, razorpayOrderId, razorpayPaymentId, razorpaySignature, status (CREATED / PAID / FAILED / REFUNDED), paidAt
bookings	paymentStatus, invoiceNumber	UNPAID → PAID flag and the assigned INV-XXXXXXXX number

- One booking can have one payment document updated in place, which keeps reconciliation simple.
- Materials are read from the booking itself, so an edited estimate is reflected in the next bill.

Screenshots

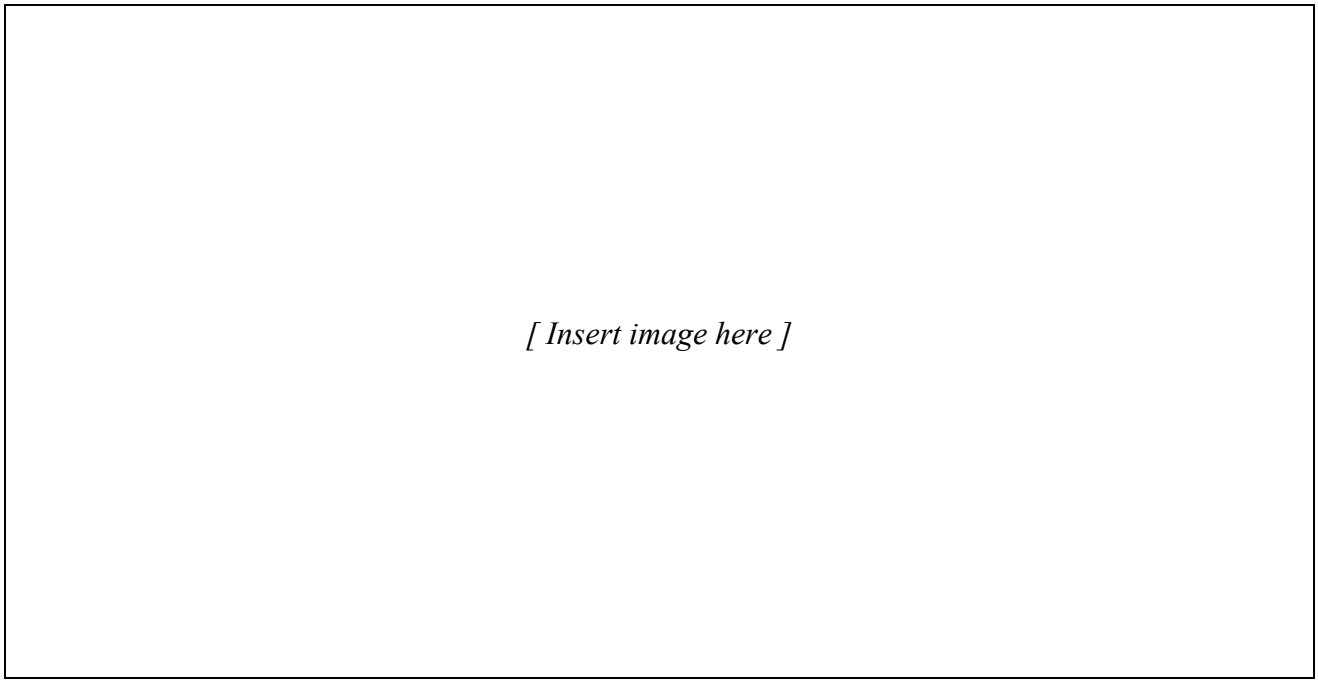


Fig 8.12: Razorpay Checkout

The customer pays the booking amount online; the server verifies the signature before the booking is marked PAID.

[Insert image here]

Fig 8.13: PDF Bill Preview & Download

The generated bill lists the service, material line items, grand total and amount in words.

8.7 REAL-TIME CHAT & NOTIFICATIONS

Purpose & Scope

Keeps both parties in sync after a booking exists. Customers and providers chat in real time inside a per-booking room, in-app notifications arrive per user, and lifecycle messages are delivered over e-mail, SMS or WhatsApp according to each user's preferences with a recorded outcome.

Functional Requirements

What the module must do:

- Socket.IO rooms: join_chat(bookingId) for the conversation and register_user(userId) for personal alerts.
- send_message persists the message and broadcasts receive_message to the room; an offline receiver additionally gets an in-app notification.
- The NotificationBell lists unread notifications with read / read-all states (notificationController).
- Lifecycle events (request, acceptance, estimate, start, completion, cancellation, message) are composed by buildLifecycleMessage and sent over the configured channels.

- Every send attempt is logged with channel, event and SENT / FAILED / SKIPPED status, honouring email / sms / whatsapp preferences; channelStatus exposes configuration health.

Out of scope: voice and video calls.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Real-time latency:** messages and notifications appear on the other side within about a second.
- **Integrity:** chat is stored per booking; only the booking's customer and provider can join its room.
- **Reliability:** a failed e-mail, SMS or WhatsApp send never blocks the booking transaction – it is logged as FAILED instead.
- **Privacy:** users opt in and out per channel through communicationPreferences.
- **CAP:** the database is authoritative; Socket.IO events are notifications of stored state.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
GET	/api/messages/:bookingId	Loads the chat history of one booking room.
GET	/api/notifications	The signed-in user's notifications, newest first.
PUT, PUT	/api/notifications/:id/read, /read-all	Marks one or all notifications read.
GET	/api/notifications/status	Which channels (e-mail, SMS, WhatsApp) are configured.
SOCKET	join_chat, send_message, register_user	Real-time events: room join, message persist + broadcast, personal alert room.

DB Design

Collection	Keys / references	Key fields
messages	bookingId, senderId, receiverId refs; index on bookingId	text, read flag
notifications	ref recipient → users; index on recipient + isRead	title, message, type, relatedBooking, isRead
bookings (embedded log)	communicationLogs[] inside the booking	channel (email / whatsapp / sms), event, status (SENT / FAILED / SKIPPED), detail, sentAt

- Keeping the delivery log inside the booking keeps a job's full communication history transactional.
- Expired or invalid channel credentials degrade to a logged SKIPPED / FAILED entry instead of an error to the user.

Screenshots

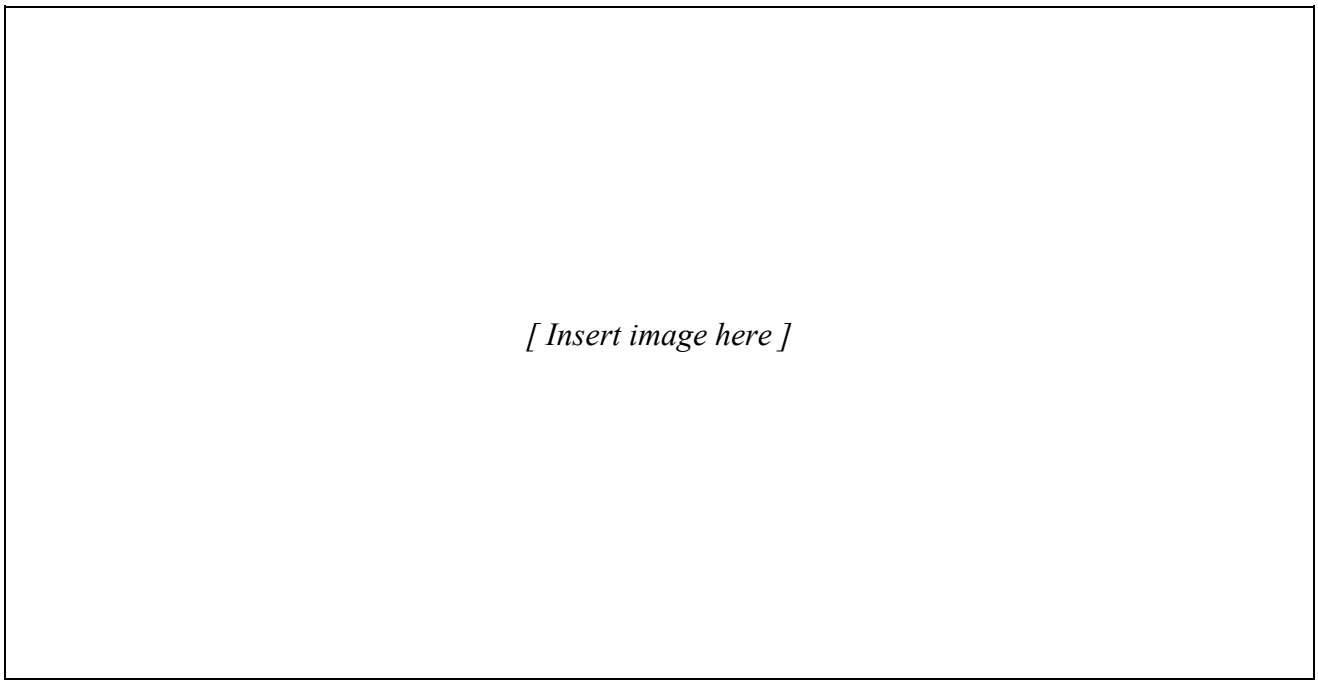


Fig 8.14: Per-Booking Chat

Customer and provider discuss the job in a real-time room tied to the booking.

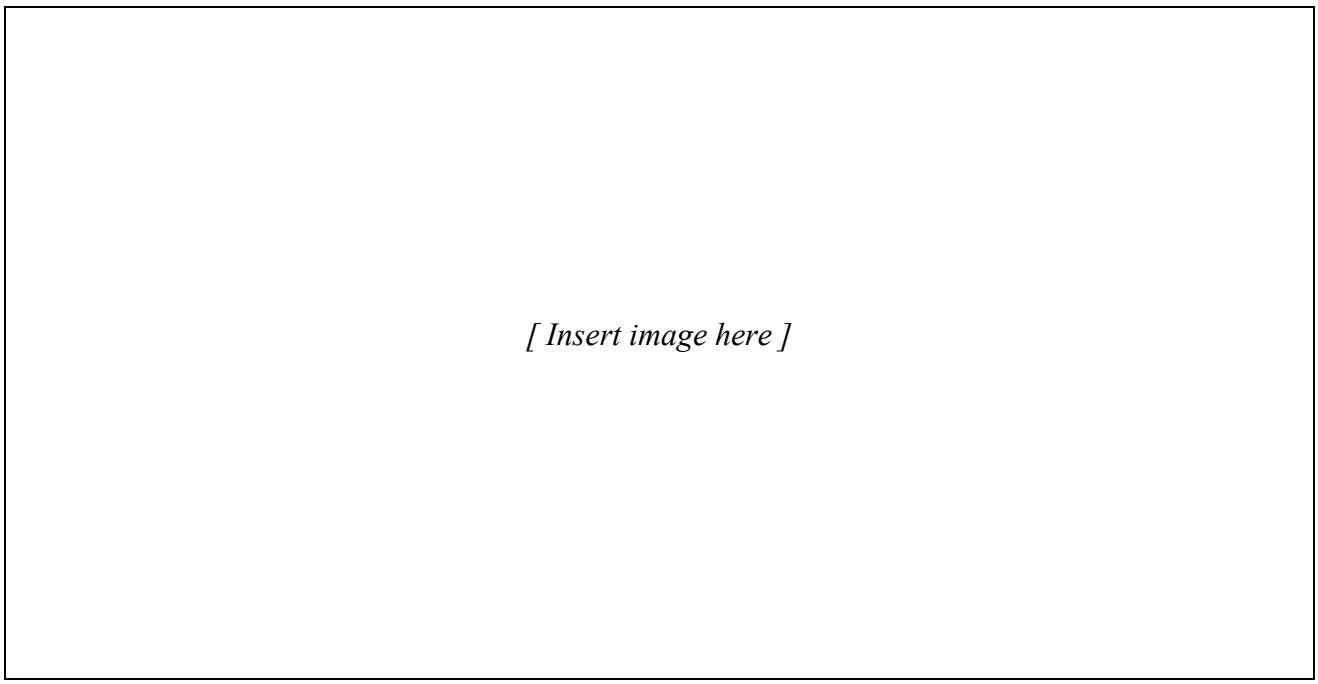


Fig 8.15: Notification Bell & Centre

In-app notifications arrive live with unread counts and one-click mark-all-read.

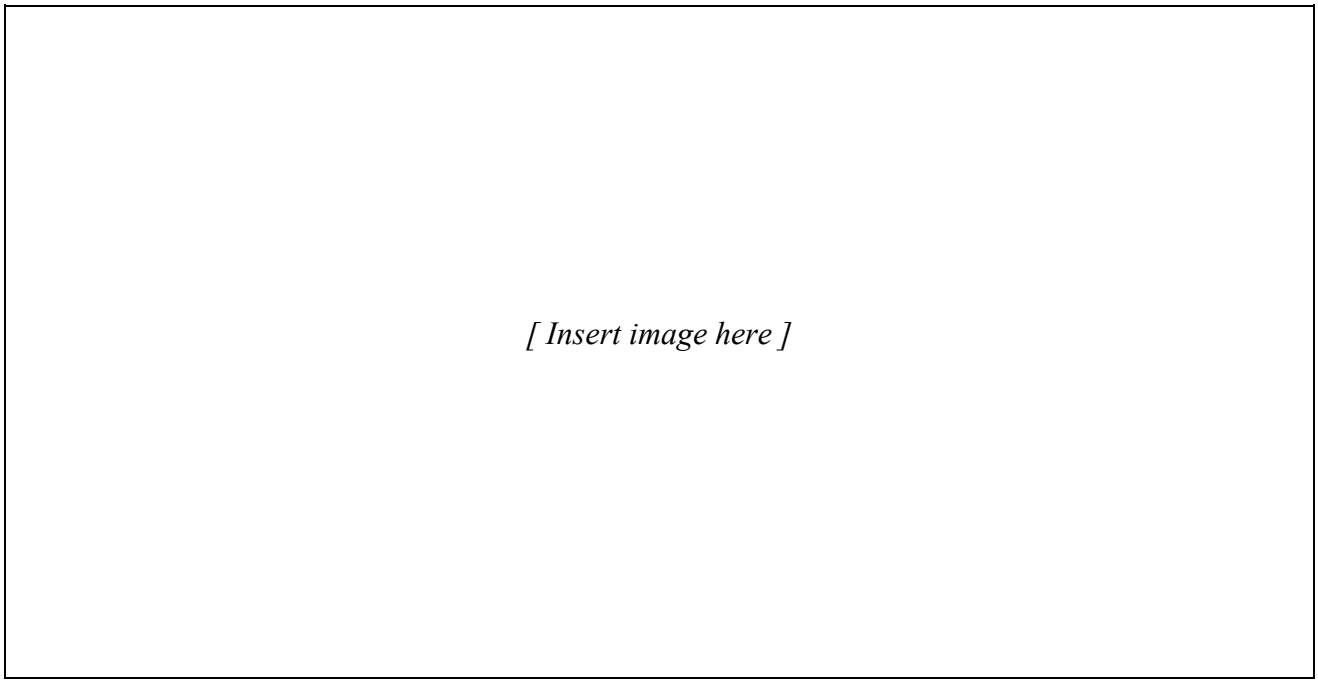


Fig 8.16: Delivery Status of Email / SMS / WhatsApp

8.8 REVIEWS, RATINGS & PROVIDER TRUST

Purpose & Scope

Turns completed work into visible reputation. After a booking is completed the customer rates the provider, service or package, and the recalculated averages appear immediately on every affected profile and listing.

Functional Requirements

What the module must do:

- A review references the customer, provider, service, package and booking and carries a 1–5 rating with a comment (reviewController.createReview).
- refreshRatings recomputes the averages on the provider (users.rating) and the service (services.rating) after every insert.
- Reviews are exposed per provider, per service and per package for the detail screens.
- A customer can delete their own review and an admin can delete any review; both recompute the averages.
- Approved-provider status, rating and portfolio together form the trust signals shown on provider cards.

Out of scope: photo reviews, anonymous ratings and review upvotes.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Integrity:** reviews reference the booking that produced them, keeping ratings tied to real work.
- **Performance:** averages are stored on the catalogue, so discovery never aggregates at read time.
- **Fairness:** only approved providers appear publicly, and every review carries its author.
- **Abuse control:** write routes pass through the same rate limiter as the rest of the API.
- **CAP:** consistency preferred – a profile rating changes are visible platform-wide on the next read.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
POST	/api/reviews	Body: booking, provider, service/package, rating 1–5, comment. Creates the review and refreshes averages.
GET	/api/reviews/provider/:providerId	Reviews shown on a provider profile.
GET	/api/reviews/service/:serviceId	Reviews shown on a service page.
GET	/api/reviews/package/:packageId	Reviews shown on a package page.
DELETE	/api/reviews/:id	Removes a review (owner or admin) and recomputes averages.

DB Design

Collection	Keys / references	Key fields
reviews	refs customer, provider, service, package, booking; index on provider / service / package	rating (1–5), comment, timestamps
users, services	rating field	Average rating updated after each review or deletion

- Reference paths are denormalised into the rating fields so profile pages stay cheap to render.
- The provider's bookingsCount on services tracks popularity alongside rating.

Screenshots

[Insert image here]

Fig 8.17: Review & Rating Form

After completion the customer rates the work; provider and service averages update immediately.

8.9 THEME, LANGUAGE & SAVED ITEMS

Purpose & Scope

Makes BuildConnect comfortable and personal: light, dark or system appearance, a searchable language switch (English, Hindi, Tamil and more through Google Translate) and a favourites list the customer can return to.

Functional Requirements

What the module must do:

- ThemeProvider applies light, dark or system mode and ThemeToggle switches it; the choice persists with the profile (users.theme).
- LanguageSelector translates the interface through the Google Translate engine and remembers the chosen language between visits.
- SavedPage keeps favourite services, packages and providers through the favourites helper.
- AppearanceSettingsPanel and GeneralSettingsPanel expose these choices inside both dashboards.
- The layout adapts through DashboardMobileNav on phones and full navigation on laptops.

Out of scope: native mobile applications and per-language content authored by hand.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Accessibility:** high-contrast light and dark themes and language choices are remembered between visits.
- **Performance:** translation and saved lists are client-side, so they add no API load.
- **Privacy:** preferences never leave the device except the theme flag stored with the profile.
- **Compatibility:** the themed layout works on phone and laptop screens alike.
- **Reliability:** a blocked translation request leaves the original English interface usable.

API Design

Representative REST routes for this module; every route is protected by the JWT middleware unless marked public.

Method	Endpoint	Purpose / contract
PUT	/api/users/profile	Stores the chosen theme with the profile so it follows the account.

DB Design

Collection	Keys / references	Key fields
users	theme field	theme (light / dark / system); language and favourites are stored on the device

- Keeping language and favourites client-side avoids synchronisation costs for anonymous browsing.

Screenshots

[Insert image here]

Fig 8.18: Language Selector

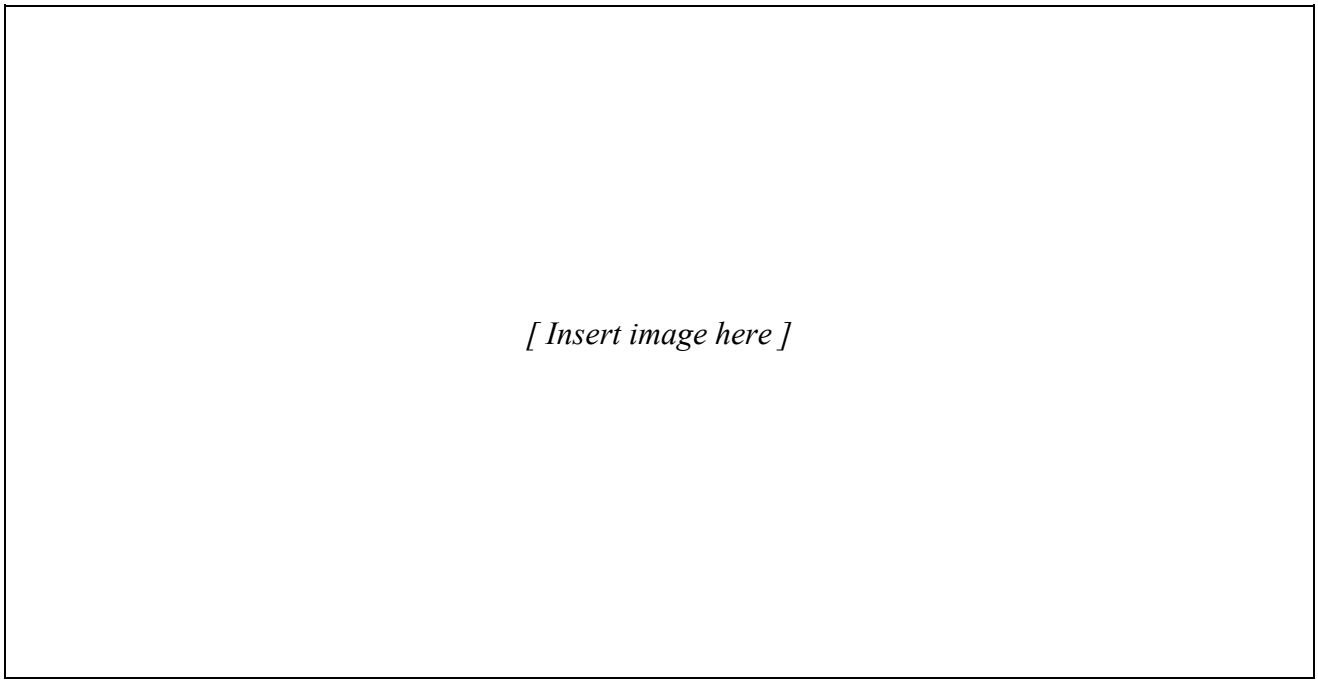


Fig 8.19: Light & Dark Themes

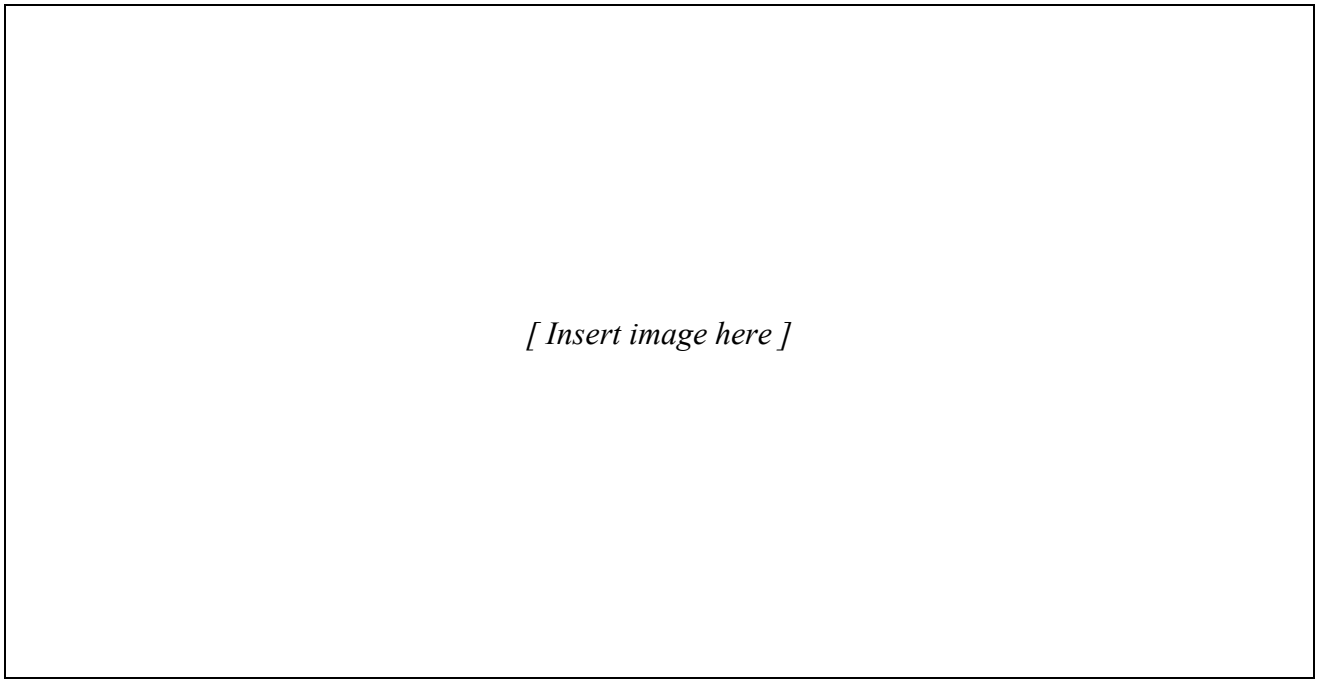


Fig 8.20: Saved Items

8.10 ADMIN OPERATIONS & TRANSPARENCY DASHBOARD

Purpose & Scope

Gives the platform administrator control and visibility: live platform metrics, provider approval, user management and content moderation over users, services, packages, bookings and reviews.

Functional Requirements

What the module must do:

- The dashboard (AdminDashboard) shows live counts of users, providers, customers, services, bookings by status and reviews.
- Pending providers are reviewed and approved (PUT /api/admin/approve-provider/:id), which unlocks their listings.
- Users, services and packages can be listed and deleted; reviews can be removed with automatic rating recalculation.
- All bookings can be inspected with customer, provider and service populated for dispute handling.
- Every admin route requires the admin role through the authorize middleware.

Out of scope: custom report builders and scheduled e-mail reports.

Non-Functional Requirements

How well the module must do it (the system-wide baseline in Section 8.0 also applies):

- **Security:** every admin route re-checks the admin role from the JWT; the dashboard aggregates only through server queries.
- **Accountability:** destructive actions (deletes, approvals) change documents directly and are visible in the affected records.
- **Performance:** metrics are count queries over indexed collections and complete in well under a second at current volumes.
- **Consistency:** impact figures are derived from the collections, never typed in manually.
- **CAP:** consistency preferred – an approval decision takes effect immediately for the whole platform.

API Design

Representative REST routes for this module; every route requires the admin role.

Method	Endpoint	Purpose / contract
GET	/api/admin/dashboard	Counts of users, providers, customers, services, bookings by status and

Method	Endpoint	Purpose / contract
		reviews.
GET	/api/admin/users	All accounts without password hashes, newest first.
PUT	/api/admin/approve-provider/:id	Approves a provider so their services become bookable.
GET	/api/admin/services, /packages, /bookings	Full platform listings populated with owner details.
GET, DELETE	/api/admin/reviews	Review moderation with automatic rating recalculation on delete.
DELETE	/api/admin/users/:id, /services/:id, /packages/:id	Content and account removal.

DB Design

Collection	Keys / references	Key fields
users	isApproved flag	Set on provider approval; role filters feed the dashboard counts
bookings, services, packages, reviews	status and reference indexes	Aggregated for dashboard counts and moderation lists

- Dashboard metrics are computed from the existing collections – no extra reporting tables are needed.
- Deleting a review recalculates both the service and the provider average in the same request.

Screenshots

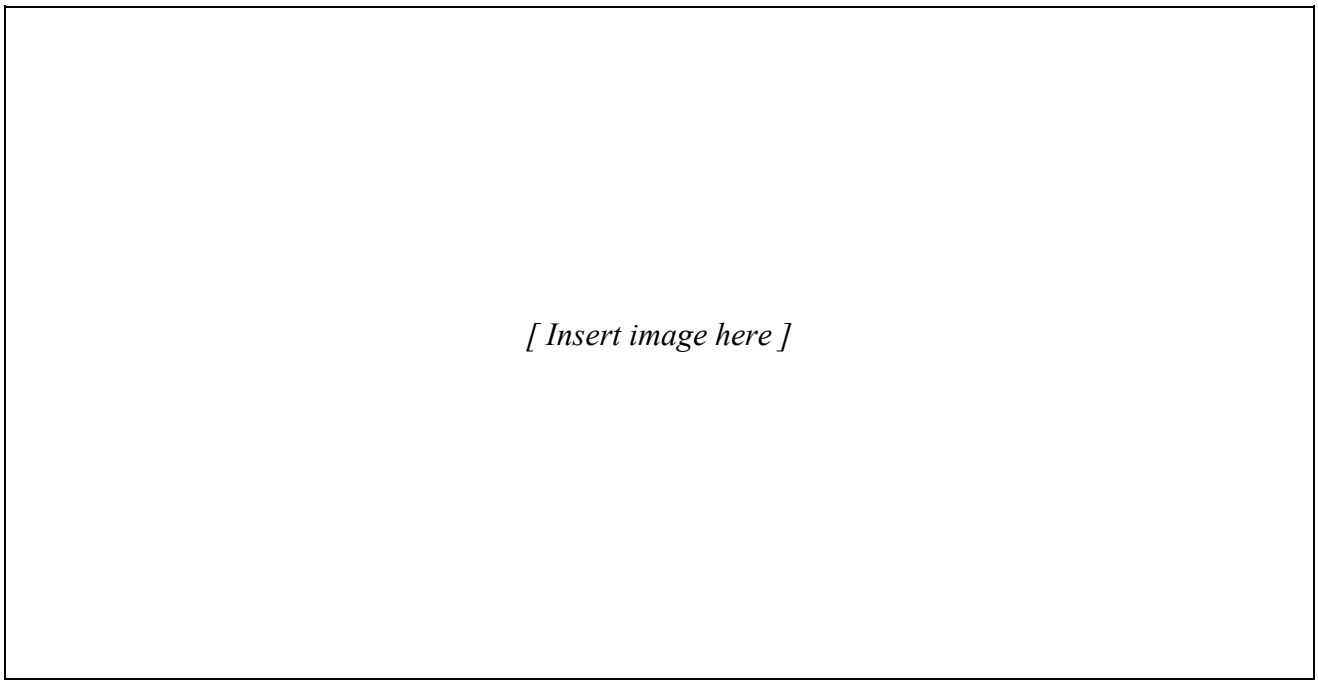


Fig 8.21: Admin Dashboard

Live counts of users, providers, services, bookings by status and reviews across the platform.

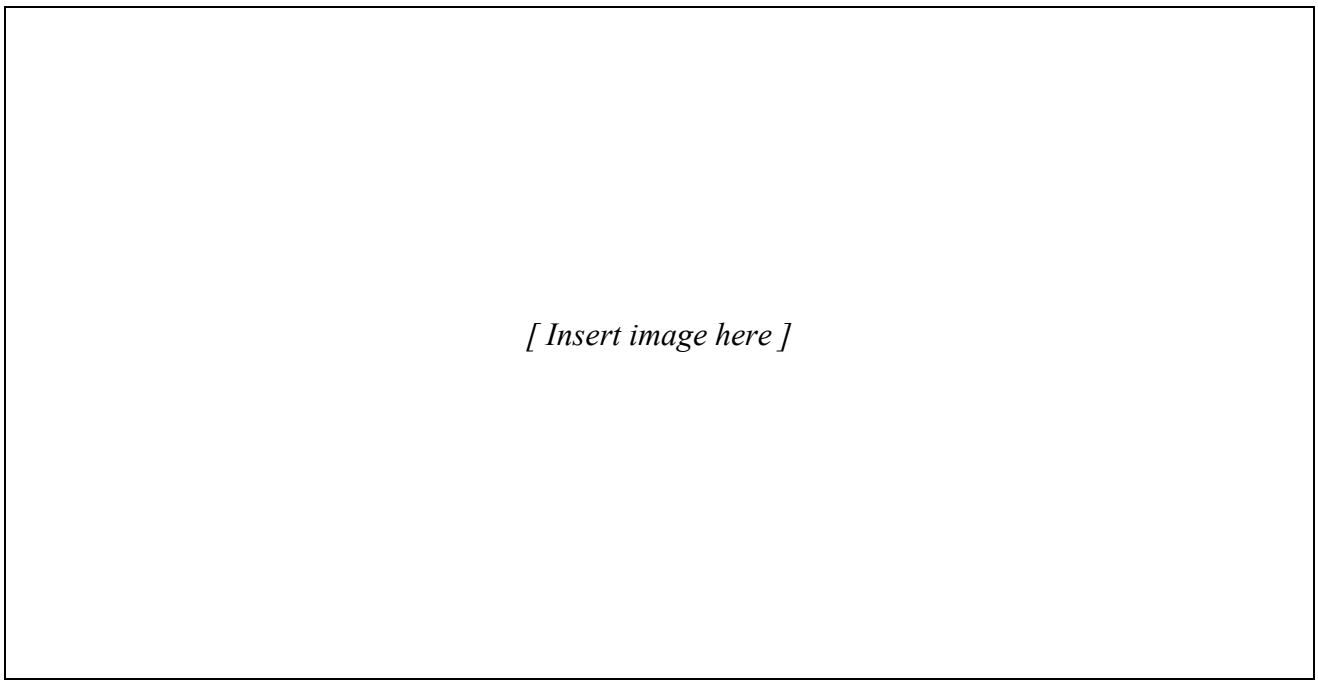


Fig 8.22: Provider Approval Workflow

The admin approves provider accounts before their services become bookable.

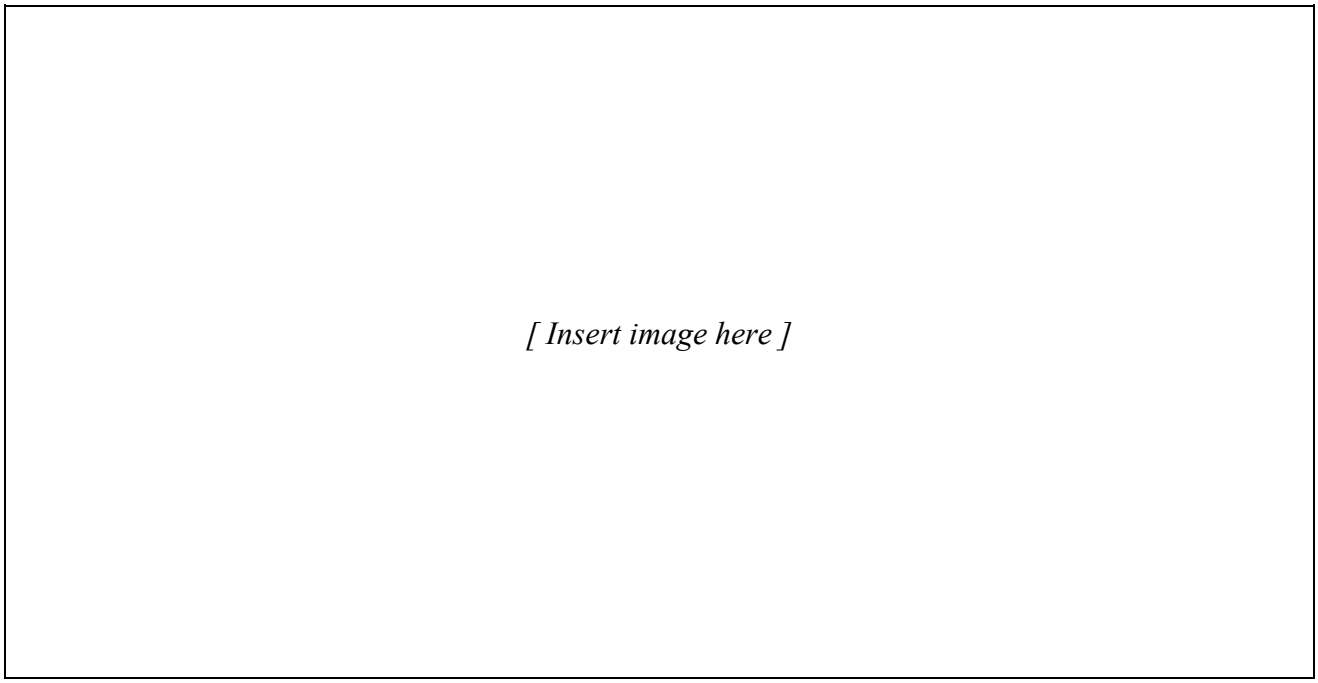


Fig 8.23: Content & User Management

CHAPTER 9
CONCLUSION

BuildConnect provides a complete digital solution for arranging home services while protecting both customers and skilled professionals. The system integrates service, package and portfolio discovery with filters and maps, slot-based bookings with a fixed status lifecycle, estimates and material tracking, Razorpay payments with server-side signature verification, numbered PDF bills, real-time per-booking chat, in-app plus e-mail, SMS and WhatsApp notifications with a delivery log, reviews with recalculated ratings, and an admin console with metrics, provider approval and content moderation within a single application.

The application was developed using React 18, Vite and Tailwind CSS for the frontend, Node.js, Express and Socket.IO for the backend, and MongoDB with the Mongoose ODM for data management, with Razorpay, Google OAuth, Nodemailer, Twilio and Leaflet integrations. Its client-server architecture supports structured communication between the user interface, the API server and the database. bcrypt password hashing, JWT authentication, role-based access, Helmet security headers, rate limiting and Zod validation protect the system, while server-side payment signature verification guarantees that no booking is marked paid without Razorpay's confirmation.

The completed project demonstrates end-to-end workflows for all three roles – Customer, Provider and Admin – with dashboards tailored to each. Its responsive interface, light and dark themes, English, Hindi and Tamil support, interactive maps and real-time chat make it accessible and pleasant to use on both phone and laptop. Its modular structure – separate controllers, schemas, routes and services – makes the application easier to maintain and allows additional features to be introduced in the future. Overall, BuildConnect successfully meets its objective of providing a secure, real-time and trustworthy platform that connects customers with skilled service providers across Tamil Nadu.

Future Enhancement

- **Provider Subscriptions** Tiered plans with boosted listing placement and lower platform fees.
- **AI Estimate Suggester** Recommend a fair quote range from historical bookings of the same category and city.
- **Dispute Resolution Centre** Structured raise-and-resolve workflow with evidence upload and admin mediation.
- **Offline Mode & Web Push** Installable PWA with an offline shell and web push notifications for bookings and messages.